P R Á C T I C A G U I A D A

La agenda de contactos

Un CRUD muy sencillo con MongoDB y Node.js

Crear, leer, modificar y borrar contactos paso a paso: primero escribiendo órdenes en la consola de MongoDB y después con pequeños programas de Node.js, todo dentro de la máquina virtual con Windows 10 en VirtualBox.

C O N T E N I D O

01 Presentación de la práctica

02 Antes de empezar

03 Parte A: el CRUD desde la consola

04 Parte B: el CRUD desde Node.js

05 Lista de comprobación final

1. Presentación de la práctica

¿Qué vamos a hacer?

Vamos a crear la agenda de contactos de una persona. En ella guardaremos el **nombre**, el **teléfono** y la **ciudad** de cada contacto. Con esos datos haremos las cuatro operaciones básicas que se pueden hacer en cualquier base de datos: crear, leer, modificar y borrar.

Primero lo haremos escribiendo órdenes en la consola de MongoDB (`mongosh`) y después repetiremos las mismas cuatro operaciones con cuatro programas cortos de Node.js.

¿Qué significa CRUD?

CRUD son las iniciales en inglés de las cuatro operaciones básicas. Cada letra se corresponde con una orden de MongoDB:

Letra	En inglés	En español	Orden de MongoDB
C	Create	Crear	`insertOne` / `insertMany`
R	Read	Leer	`find` / `findOne` / `countDocuments`
U	Update	Actualizar	`updateOne`
D	Delete	Borrar	`deleteOne`

Cómo está organizada

Parte	Contenido	Ejercicios
Antes de empezar	Abrir la máquina virtual y comprobar los programas	1
Parte A	Crear, leer, modificar y borrar desde la consola `mongosh`	2 a 7
Parte B	Crear, leer, modificar y borrar desde programas de Node.js	8 a 12

Tiempo aproximado: entre 1 hora y media y 2 horas.

Qué tienes que entregar

- Este documento con una captura de pantalla en el espacio reservado de cada ejercicio.
- La carpeta del proyecto de Node.js **sin** la carpeta `node_modules` (apartado B.7).

Cómo hacer las capturas

Dentro de la ventana de la máquina virtual usa el menú **Ver > Tomar captura de pantalla** (o la combinación **Ctrl derecho + E**). Elige dónde guardar la imagen y después pégala en el espacio del ejercicio correspondiente.

También puedes pulsar **Windows + Mayús + S** dentro de la máquina virtual, seleccionar la zona de la pantalla y pegarla directamente en el documento con **Ctrl + V**.

2. Antes de empezar

Esta práctica usa la misma máquina virtual de la práctica de la biblioteca: **Win10-MongoDB**. En ella ya están instalados MongoDB, la consola `mongosh`, Node.js y Visual Studio Code, así que no hay que instalar nada nuevo.

Abrir la máquina virtual y la terminal

Paso 1. En tu ordenador real abre **VirtualBox**.

Paso 2. En la lista de la izquierda selecciona la máquina **Win10-MongoDB** y pulsa **Iniciar** (la flecha verde).

Paso 3. Espera a que aparezca el escritorio de Windows 10.

Paso 4. Dentro de la máquina virtual pulsa la tecla **Windows**, escribe `cmd` y haz clic en **Símbolo del sistema**.

Paso 5. Se abre una ventana negra con el texto `C:\Users\alumno>`. Ahí escribirás las órdenes y pulsarás **Intro** para ejecutarlas.

Tres palabras que vas a usar

Palabra	Qué es	En esta práctica
Base de datos	El contenedor general donde se guarda todo.	`agenda`
Colección	Un grupo de elementos del mismo tipo.	`contactos`
Documento	Un elemento concreto con sus datos.	Un contacto

Así es un contacto en MongoDB

> UN DOCUMENTO
> `{ nombre: "Ana López", telefono: "600111222", ciudad: "Valencia" }`

- Todo el documento va entre llaves `{ }`.
- Cada dato se escribe como `campo: valor` y los datos se separan con comas.
- Los textos van entre comillas rectas `"`. El teléfono también lo escribimos entre comillas porque es un texto: no vamos a hacer cuentas con él.
- MongoDB distingue mayúsculas y minúsculas: `nombre` y `Nombre` serían dos campos distintos.
- Las órdenes se escriben exactamente así: `insertOne`, `find`, `updateOne`, `deleteOne`, con la mayúscula en su sitio.

Ejercicio 1 Comprueba que todo está listo

En el Símbolo del sistema de la máquina virtual escribe estas órdenes una detrás de otra, pulsando **Intro** después de cada una.

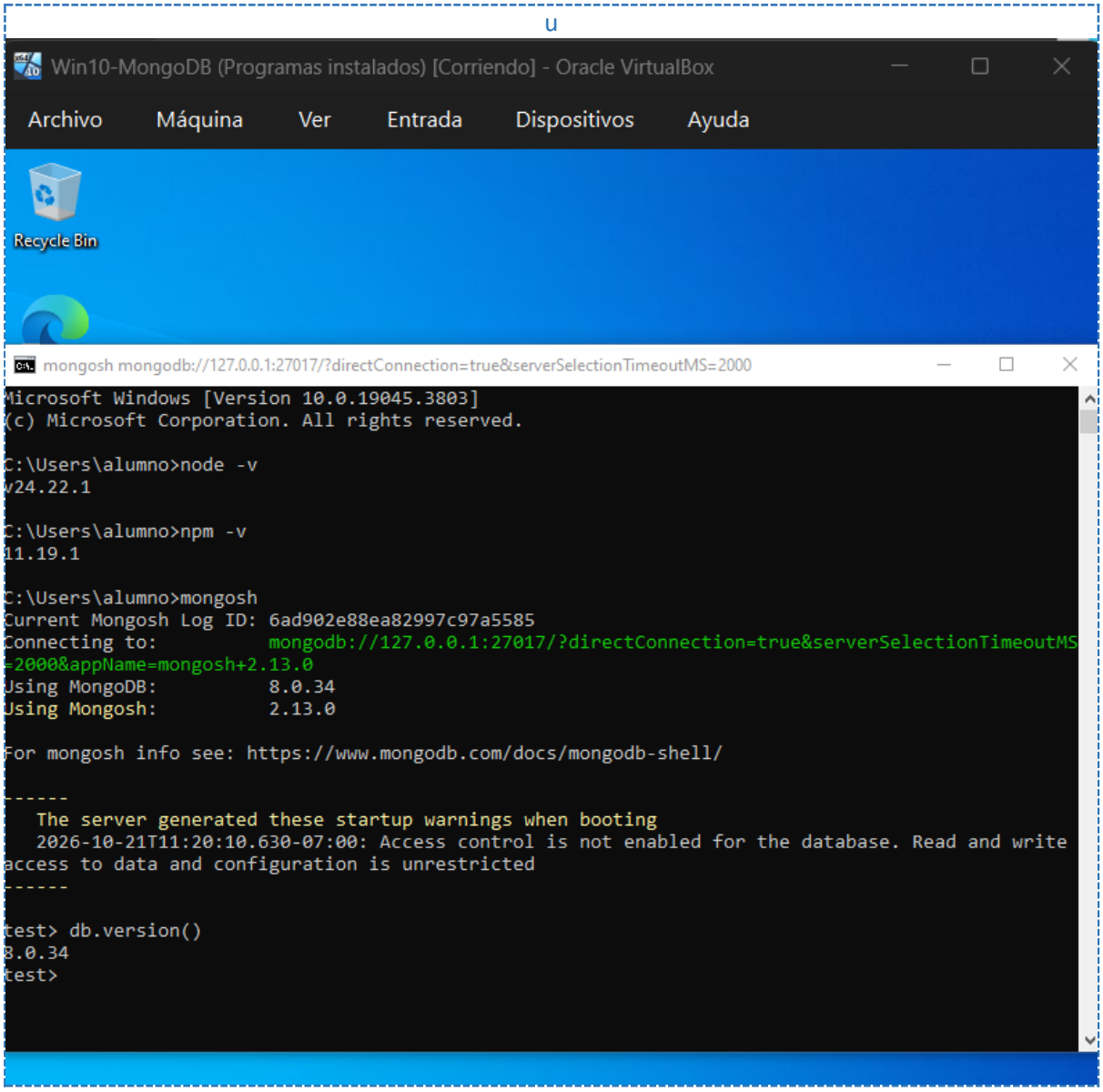
Cada orden debe responder con un número de versión. Haz una captura donde se vean las tres versiones (Node.js, npm y MongoDB).

ÓRDENES

```
node -v
npm -v
mongosh
db.version()
exit
```

Orden	Qué hace
`node -v`	Muestra la versión de Node.js instalada.
`npm -v`	Muestra la versión de npm, el programa que descarga librerías.
`mongosh`	Abre la consola de MongoDB. Al final aparece el indicador `test>`.
`db.version()`	Dentro de mongosh, muestra la versión del servidor de MongoDB.
`exit`	Cierra mongosh y vuelve al Símbolo del sistema.

u


Win10-MongoDB (Programas instalados) [Corriendo] - Oracle VirtualBox
Archivo
Máquina
Ver
Entrada
Dispositivos
Ayuda
Recycle Bin
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.
C:\Users\alumno>node -v
v24.22.1
C:\Users\alumno>npm -v
11.19.1
C:\Users\alumno>mongosh
Current Mongosh Log ID: 6ad902e88ea82997c97a5585
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS
=2000&appName=mongosh+2.13.0
Using MongoDB: 8.0.34
Using Mongosh: 2.13.0
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
The server generated these startup warnings when booting
2026-10-21T11:20:10.630-07:00: Access control is not enabled for the database. Read and write
access to data and configuration is unrestricted
test> db.version()
8.0.34
test>

3. Parte A: el CRUD desde la consola

En el Símbolo del sistema escribe `mongosh` y pulsa **Intro**. Cuando veas el indicador `test>` ya puedes escribir las órdenes de esta parte.

A.1 Crear (C): insertar contactos

Para elegir la base de datos se escribe `use` seguido de su nombre. Si la base de datos no existe, MongoDB la crea sola en cuanto guardamos el primer documento.

Para guardar **un** documento se usa `insertOne`. La orden siempre tiene la forma `db.colección.insertOne({ datos })`.

Ejercicio 2 Crear la agenda y el primer contacto

Entra en la base de datos `agenda`, guarda el contacto de Ana López y comprueba que se ha guardado.

Paso 1. Escribe `use agenda` y pulsa **Intro**. MongoDB responde `switched to db agenda` y el indicador cambia a `agenda>`.

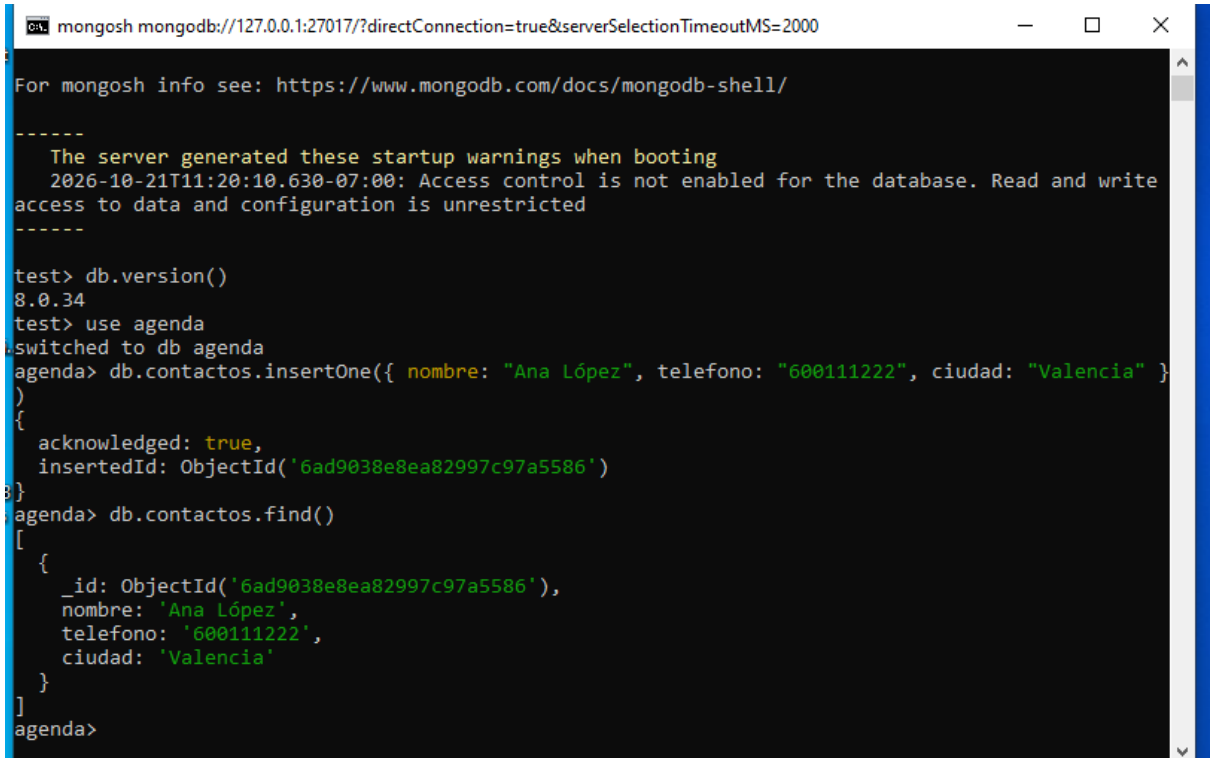
Paso 2. Escribe la orden de inserción que aparece debajo y pulsa **Intro**.

Paso 3. Mira la respuesta: `acknowledged: true` significa que MongoDB ha guardado el contacto, e `insertedId` es el identificador único que MongoDB ha creado solo en el campo `_id`.

Paso 4. Escribe `db.contactos.find()` y pulsa **Intro** para ver todos los contactos. Debe aparecer Ana López con su `_id`.

ÓRDENES

```
use agenda
db.contactos.insertOne({ nombre: "Ana López", telefono: "600111222", ciudad:
"Valencia" })
db.contactos.find()
```



Espacio para la captura del ejercicio 2

Para guardar **varios** documentos de una sola vez se usa `insertMany`. Todos los documentos van dentro de unos corchetes `[ ]` y separados por comas.

Ejercicio 3 Añadir varios contactos a la vez

Inserta de una sola vez los cuatro contactos siguientes en la colección `contactos`.

Paso 1. Abre el **Bloc de notas** dentro de la máquina virtual y escribe en él la orden completa que aparece debajo. Revisa que se cierran todas las llaves `{ }` y el corchete `]`.

Paso 2. Selecciona todo el texto (**Ctrl + E**) y cópialo (**Ctrl + C**).

Paso 3. Vuelve a la ventana de mongosh y pega con el **botón derecho del ratón**. Mientras la orden no está terminada, mongosh muestra tres puntos `...` al principio de cada línea.

Paso 4. Pulsa **Intro**. En la respuesta aparecen cuatro identificadores, numerados del `'0'` al `'3'`: uno por cada contacto guardado.

ÓRDENES

```
db.contactos.insertMany([
  { nombre: "Luis Martín", telefono: "600333444", ciudad: "Madrid" },
  { nombre: "Marta Ruiz", telefono: "600555666", ciudad: "Valencia" },
  { nombre: "Pedro Gómez", telefono: "600777888", ciudad: "Sevilla" },
  { nombre: "Lucía Sanz", telefono: "600999000", ciudad: "Barcelona" }
])
```

Espacio



para la captura del ejercicio 3

A.2 Leer (R): consultar contactos

`find()` busca documentos. Si los paréntesis están vacíos, devuelve todos. `countDocuments()` cuenta cuántos documentos hay en la colección.

Ejercicio 4 Ver todos los contactos

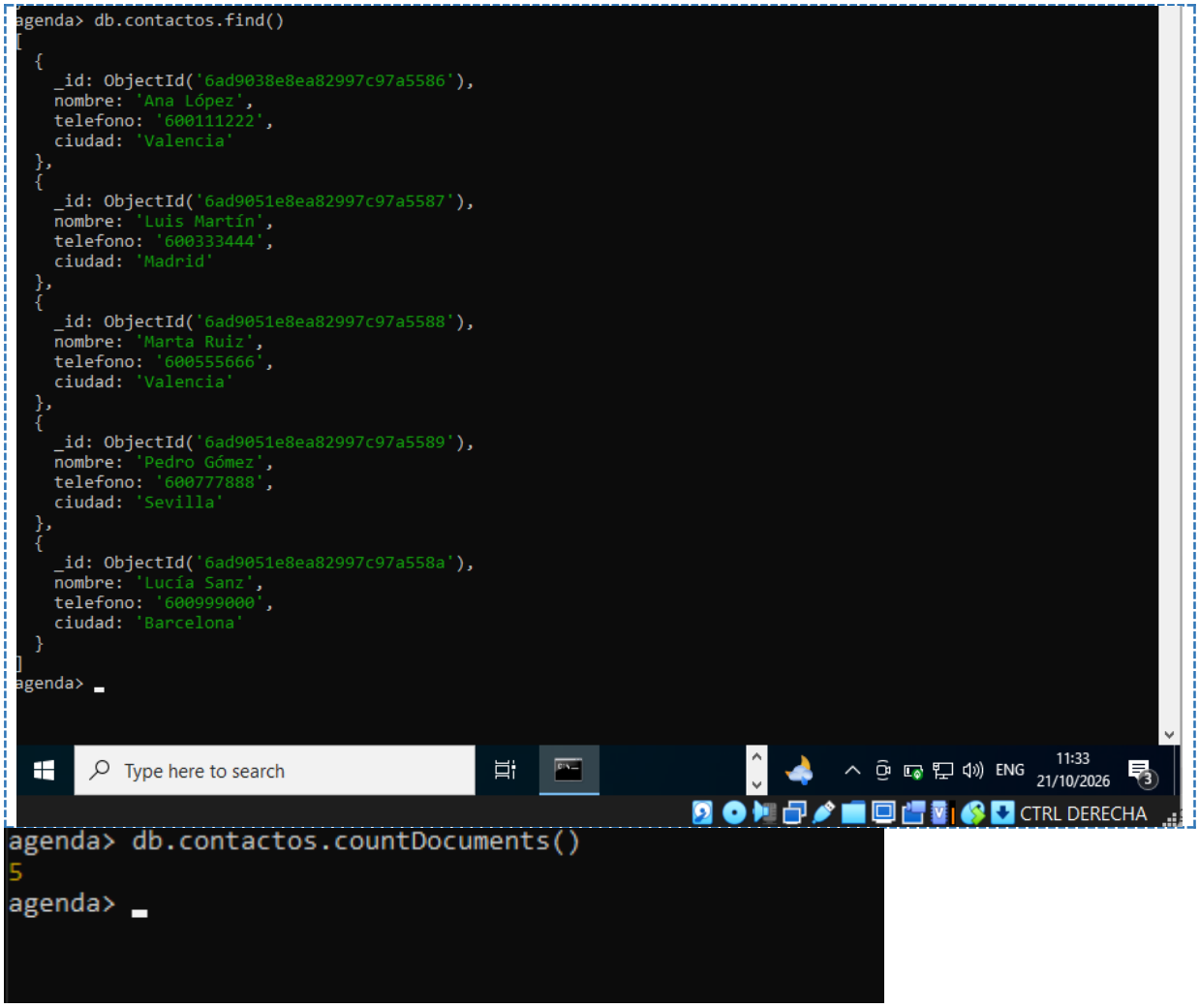
Muestra todos los contactos de la agenda y después cuéntalos. Debe salir el número **5**.

Paso 1. Escribe `db.contactos.find()` y pulsa **Intro**. Aparecen los cinco contactos, cada uno entre llaves.

Paso 2. Escribe `db.contactos.countDocuments()` y pulsa **Intro**. Aparece el número `5`.

ÓRDENES

```
db.contactos.find()
db.contactos.countDocuments()
```



Si dentro de `find()` ponemos un **filtro**, solo aparecen los documentos que lo cumplen. El filtro se escribe igual que un dato: `{ campo: valor }`.

Además, se puede añadir un segundo par de llaves para elegir qué campos se ven: `1` para mostrar un campo y `0` para ocultarlo. El campo `_id` se muestra siempre salvo que pongamos `_id: 0`.

Ejercicio 5 Buscar los contactos de una ciudad

Muestra los contactos que viven en **Valencia**. Deben aparecer dos: Ana López y Marta Ruiz.

Después repite la búsqueda mostrando solo el nombre y el teléfono.

Paso 1. Escribe la primera orden de debajo y pulsa **Intro**. Aparecen los dos contactos completos.

Paso 2. Escribe la segunda orden y pulsa **Intro**. Ahora solo aparecen el nombre y el teléfono, sin el `_id` ni la ciudad.

ÓRDENES

```
db.contactos.find({ ciudad: "Valencia" })
db.contactos.find({ ciudad: "Valencia" }, { nombre: 1, telefono: 1, _id: 0 })
```

```
agenda> db.contactos.find({ ciudad: "Valencia" })
[
  {
    _id: ObjectId('6ad9038e8ea82997c97a5586'),
    nombre: 'Ana López',
    telefono: '600111222',
    ciudad: 'Valencia'
  },
  {
    _id: ObjectId('6ad9051e8ea82997c97a5588'),
    nombre: 'Marta Ruiz',
    telefono: '600555666',
    ciudad: 'Valencia'
  }
]
agenda>
```

```
agenda> db.contactos.find({ ciudad: "Valencia" }, { nombre: 1, telefono: 1, _id: 0 })
[
  { nombre: 'Ana López', telefono: '600111222' },
  { nombre: 'Marta Ruiz', telefono: '600555666' }
]
agenda>
```

A.3 Actualizar (U): modificar un contacto

`updateOne` modifica el primer documento que cumple un filtro. Recibe dos partes separadas por una coma:

- La primera indica **a quién** cambiar: `{ nombre: "Luis Martín" }`.
- La segunda indica **qué** cambiar, siempre dentro de `$set`: `{ $set: { telefono: "611222333" } }`.

Ejercicio 6 Cambiar un teléfono

Luis Martín ha cambiado de número. Su nuevo teléfono es **611222333**. Modifícalo y comprueba el cambio.

Paso 1. Escribe la orden `updateOne` de debajo y pulsa **Intro**.

Paso 2. Mira la respuesta: `matchedCount: 1` significa que ha encontrado un contacto con ese nombre y `modifiedCount: 1` que lo ha modificado.

Paso 3. Escribe la orden `findOne` y pulsa **Intro**. `findOne` devuelve un solo documento: el de Luis, con el teléfono nuevo.

ÓRDENES

```
db.contactos.updateOne({ nombre: "Luis Martín" }, { $set: { telefono: "611222333" } })
db.contactos.findOne({ nombre: "Luis Martín" })
```

```
agenda> db.contactos.updateOne({ nombre: "Luis Martín" }, { $set: { telefono: "611222333" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
agenda>
```

Espacio para la captura del ejercicio 6

```
agenda> db.contactos.findOne({ nombre: "Luis Martín" })
{
  _id: ObjectId('6ad9051e8ea82997c97a5587'),
  nombre: 'Luis Martín',
  telefono: '611222333',
  ciudad: 'Madrid'
}
agenda>
```

A.4 Borrar (D): eliminar un contacto

`deleteOne` borra el primer documento que cumple el filtro. Solo necesita una parte: **a quién** borrar.

Ejercicio 7 Borrar un contacto

Borra a Pedro Gómez de la agenda, cuenta los contactos que quedan (deben ser **4**) y muestra sus nombres.

Paso 1. Escribe la orden `deleteOne` y pulsa **Intro**. La respuesta `deletedCount: 1` indica que se ha borrado un contacto.

Paso 2. Escribe `db.contactos.countDocuments()` y pulsa **Intro**. Aparece el número `4`.

Paso 3. Escribe la orden `find` que muestra solo los nombres y pulsa **Intro**. Pedro Gómez ya no aparece.

Paso 4. Escribe `exit` y pulsa **Intro** para salir de mongosh.

ÓRDENES

```
db.contactos.deleteOne({ nombre: "Pedro Gómez" })
db.contactos.countDocuments()
db.contactos.find({}, { nombre: 1, _id: 0 })
exit
```

```
agenda> db.contactos.deleteOne({ nombre: "Pedro Gómez" })
{ acknowledged: true, deletedCount: 1 }
agenda>
```

```
agenda> db.contactos.countDocuments()
4
agenda>
```

```
agenda> db.contactos.find({}, { nombre: 1, _id: 0 })
[
  { nombre: 'Ana López' },
  { nombre: 'Luis Martín' },
  { nombre: 'Marta Ruiz' },
  { nombre: 'Lucía Sanz' }
]
agenda>
```

Espacio para la

Cómo queda la agenda al terminar la Parte A

Nombre	Teléfono	Ciudad
Ana López	600111222	Valencia
Luis Martín	611222333	Madrid
Marta Ruiz	600555666	Valencia
Lucía Sanz	600999000	Barcelona

4. Parte B: el CRUD desde Node.js

Ahora haremos las mismas cuatro operaciones con programas. Cada operación será un archivo `.js` que ejecutaremos con la orden `node`, siempre dentro de la máquina virtual.

B.1 Crear el proyecto

Ejercicio 8 Preparar la carpeta del proyecto

Crea en el Escritorio la carpeta `agenda-contactos`, conviértela en un proyecto de Node.js e instala la librería de MongoDB.

Haz una captura donde se vean `package.json` y `node_modules` dentro de la carpeta.

Paso 1. Abre un **Símbolo del sistema** nuevo (tecla **Windows**, escribe `cmd`).

Paso 2. Ve al Escritorio: escribe `cd Desktop` y pulsa **Intro**.

Paso 3. Crea la carpeta: escribe `mkdir agenda-contactos` y pulsa **Intro**. En el Escritorio aparece una carpeta nueva.

Paso 4. Entra en ella: escribe `cd agenda-contactos` y pulsa **Intro**. La ruta del indicador termina ahora en `agenda-contactos>`.

Paso 5. Crea el proyecto: escribe `npm init -y` y pulsa **Intro**. Se crea el archivo `package.json`, que describe el proyecto.

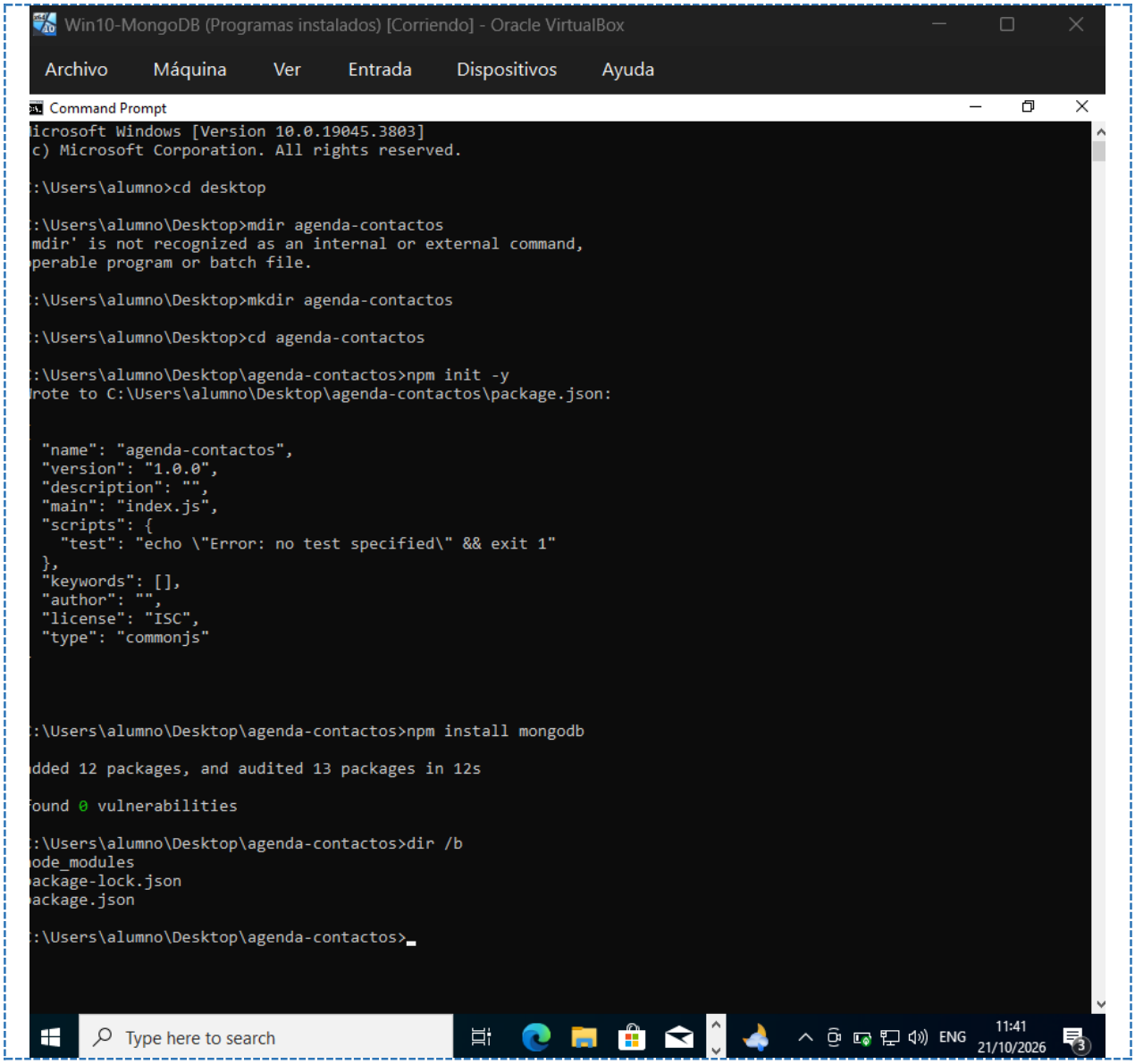
Paso 6. Instala la librería de MongoDB: escribe `npm install mongodb` y pulsa **Intro**. Se crean la carpeta `node_modules` y el archivo `package-lock.json`.

Paso 7. Comprueba lo que hay en la carpeta: escribe `dir /b` y pulsa **Intro**. Deben aparecer `node_modules`, `package-lock.json` y `package.json`.

Paso 8. Abre la carpeta en Visual Studio Code: escribe `code .` (con el punto) y pulsa **Intro**. Si VS Code pregunta si confías en los autores de la carpeta, pulsa **Sí, confío en los autores**.

ÓRDENES

```
cd Desktop
mkdir agenda-contactos
cd agenda-contactos
npm init -y
npm install mongodb
dir /b
code .
```



B.2 La plantilla de todos los programas

Los cuatro programas tienen la misma estructura. Solo cambia la parte central, la que está entre `const contactos = ...` y `} catch`:

PLANTILLA

```
const { MongoClient } = require("mongodb");
const cliente = new MongoClient("mongodb://127.0.0.1:27017");

async function main() {
  try {
    await cliente.connect();
    const contactos = cliente.db("agenda").collection("contactos");

    // ===== AQUÍ VA EL CÓDIGO DE CADA EJERCICIO =====

  } catch (error) {
    console.log("Error: " + error.message);
  } finally {
    await cliente.close();
  }
}

main();
```

Parte	Explicación
`require("mongodb")`	Carga la librería que instalaste con npm.
`new MongoClient(...)`	Prepara la conexión con el MongoDB de la máquina virtual (dirección `127.0.0.1`, puerto `27017`).
`async` / `await`	`await` espera a que MongoDB responda antes de seguir. Solo se puede usar dentro de una función `async`.
`cliente.db("agenda").collection("contactos")`	Elige la base de datos `agenda` y la colección `contactos`, las mismas de la Parte A.
`try` / `catch` / `finally`	`try`: lo que queremos hacer. `catch`: qué hacer si hay un fallo. `finally`: se ejecuta siempre; ahí se cierra la conexión.
`console.log(...)`	Escribe un mensaje en la terminal. Con el signo `+` se unen textos y datos.
`main();`	La última línea pone en marcha el programa.

B.3 Crear (C): crear.js

Ejercicio 9 crear.js

Crea el programa `crear.js` que añada a la agenda el contacto **Sara Díaz**, teléfono **622444555**, ciudad **Alicante**.

El programa debe mostrar el identificador del contacto nuevo y cuántos contactos hay ahora (deben ser **5**). Ejecútalo **una sola vez**: cada ejecución añade otro contacto igual.

Paso 1. En VS Code, en el panel izquierdo (Explorador), pasa el ratón por encima del nombre de la carpeta **AGENDA-CONTACTOS** y pulsa el botón **Nuevo archivo** (o usa el menú **Archivo > Nuevo archivo de texto**).

Paso 2. Escribe el nombre `crear.js` y pulsa Intro. El archivo se abre vacío en el centro de la pantalla.

Paso 3. Escribe (o copia y pega) el código completo que aparece debajo.

Paso 4. Guarda el archivo con **Ctrl + S**. Si en la pestaña del archivo aparece un punto blanco, es que aún no está guardado.

Paso 5. Abre la terminal de VS Code con **Ctrl + ñ** (o menú **Terminal > Nuevo terminal**). Ya está situada en la carpeta del proyecto.

Paso 6. Escribe `node crear.js` y pulsa Intro.

CREAR.JS

```
const { MongoClient } = require("mongodb");
const cliente = new MongoClient("mongodb://127.0.0.1:27017");

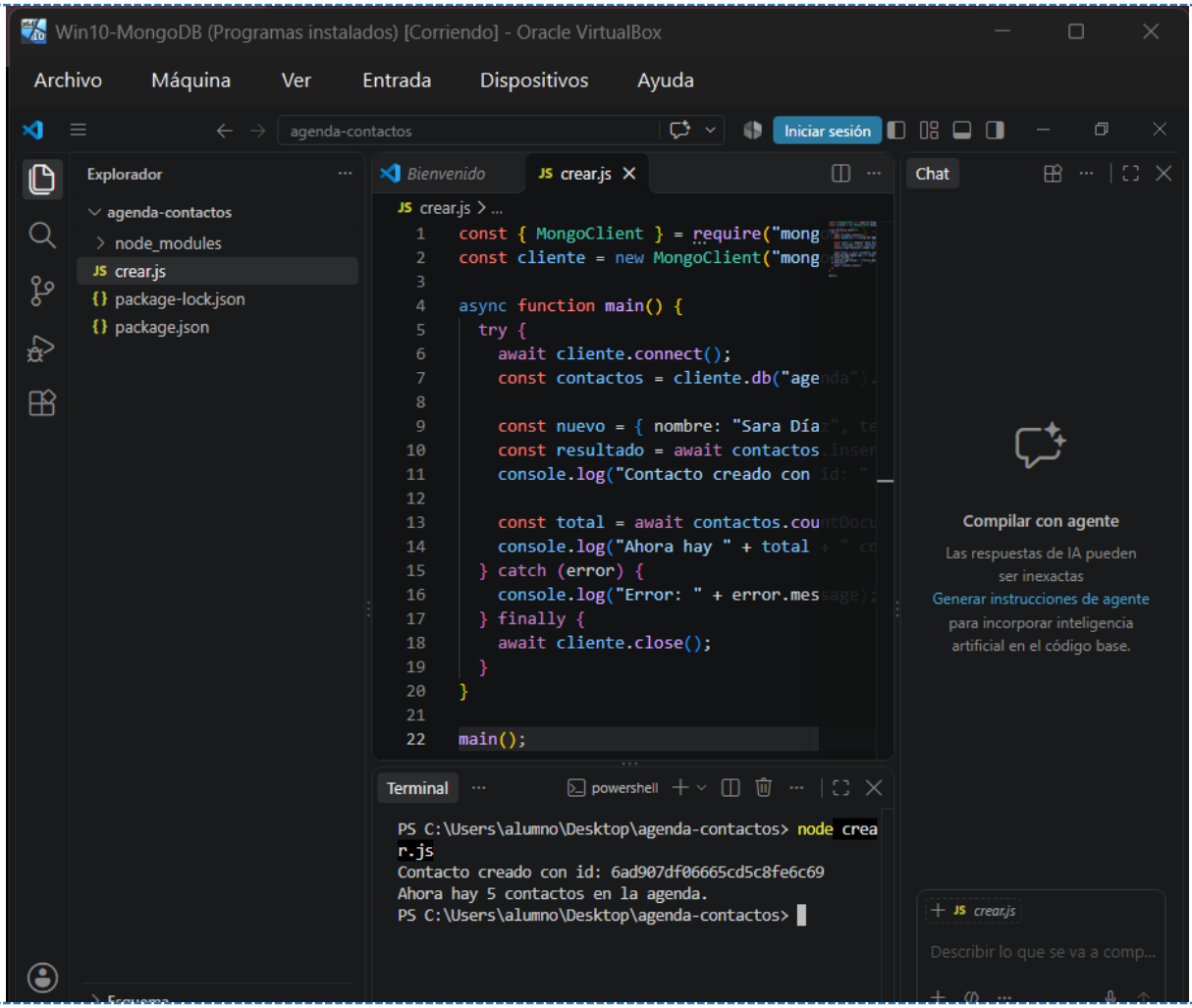
async function main() {
  try {
    await cliente.connect();
    const contactos = cliente.db("agenda").collection("contactos");

    const nuevo = { nombre: "Sara Díaz", telefono: "622444555", ciudad: "Alicante"
};
    const resultado = await contactos.insertOne(nuevo);
    console.log("Contacto creado con id: " + resultado.insertedId);

    const total = await contactos.countDocuments();
    console.log("Ahora hay " + total + " contactos en la agenda.");
  } catch (error) {
    console.log("Error: " + error.message);
  } finally {
    await cliente.close();
  }
}

main();
```

Línea	Qué hace
`const nuevo = { ... }`	Guarda en la variable `nuevo` los datos del contacto.
`await contactos.insertOne(nuevo)`	Inserta el contacto, igual que en mongosh. La respuesta se guarda en `resultado`.
`resultado.insertedId`	Es el `_id` que MongoDB ha creado para el contacto.
`contactos.countDocuments()`	Cuenta los contactos que hay ahora en la colección.



B.4 Leer (R): leer.js

En Node.js, `find()` no devuelve directamente los documentos: hay que añadir `.toArray()` para obtener una lista. Después se recorre la lista con un bucle `for`, que repite las mismas instrucciones para cada contacto.

Ejercicio 10 leer.js

Crea el programa `leer.js` que muestre todos los contactos con este formato:

`Ana López - 600111222 (Valencia)`

Al final debe mostrar el total de contactos.

Paso 1. En VS Code, en el panel izquierdo (Explorador), pasa el ratón por encima del nombre de la carpeta **AGENDA-CONTACTOS** y pulsa el botón **Nuevo archivo** (o usa el menú **Archivo > Nuevo archivo de texto**).

Paso 2. Escribe el nombre `leer.js` y pulsa Intro. El archivo se abre vacío en el centro de la pantalla.

Paso 3. Escribe (o copia y pega) el código completo que aparece debajo.

Paso 4. Guarda el archivo con **Ctrl + S**. Si en la pestaña del archivo aparece un punto blanco, es que aún no está guardado.

Paso 5. Abre la terminal de VS Code con **Ctrl + ñ** (o menú **Terminal > Nuevo terminal**). Ya está situada en la carpeta del proyecto.

Paso 6. Escribe `node leer.js` y pulsa Intro.

LEER.JS

```
const { MongoClient } = require("mongodb");
const cliente = new MongoClient("mongodb://127.0.0.1:27017");

async function main() {
  try {
    await cliente.connect();
    const contactos = cliente.db("agenda").collection("contactos");

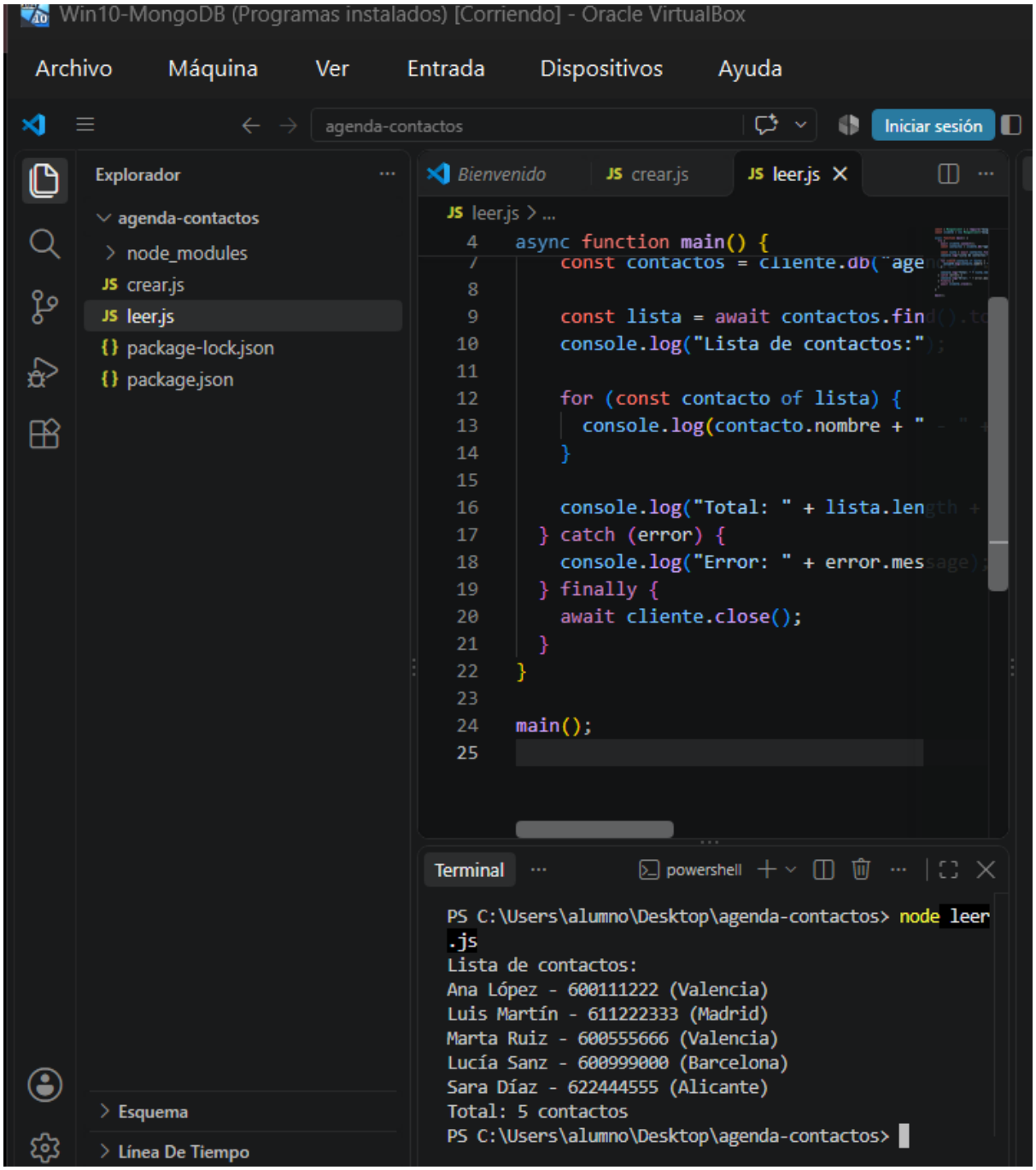
    const lista = await contactos.find().toArray();
    console.log("Lista de contactos:");

    for (const contacto of lista) {
      console.log(contacto.nombre + " - " + contacto.telefono + " (" +
contacto.ciudad + ")");
    }

    console.log("Total: " + lista.length + " contactos");
  } catch (error) {
    console.log("Error: " + error.message);
  } finally {
    await cliente.close();
  }
}

main();
```

Línea	Qué hace
`contactos.find().toArray()`	Busca todos los contactos y los guarda como una lista en la variable `lista`.
`for (const contacto of lista)`	Recorre la lista: en cada vuelta, `contacto` es uno de los contactos.
`contacto.nombre`	Con un punto se accede a cada dato del contacto: `nombre`, `telefono`, `ciudad`.
`lista.length`	Es el número de elementos que tiene la lista.



Espacio para la captura del ejercicio 10

B.5 Actualizar (U): actualizar.js

Ejercicio 11 actualizar.js

Marta Ruiz se ha mudado a **Castellón**. Crea el programa `actualizar.js` que cambie su ciudad.

El programa debe mostrar cuántos contactos se han modificado y, después, la nueva ciudad de Marta.

Paso 1. En VS Code, en el panel izquierdo (Explorador), pasa el ratón por encima del nombre de la carpeta **AGENDA-CONTACTOS** y pulsa el botón **Nuevo archivo** (o usa el menú **Archivo > Nuevo archivo de texto**).

Paso 2. Escribe el nombre `actualizar.js` y pulsa Intro. El archivo se abre vacío en el centro de la pantalla.

Paso 3. Escribe (o copia y pega) el código completo que aparece debajo.

Paso 4. Guarda el archivo con **Ctrl + S**. Si en la pestaña del archivo aparece un punto blanco, es que aún no está guardado.

Paso 5. Abre la terminal de VS Code con **Ctrl + ñ** (o menú **Terminal > Nuevo terminal**). Ya está situada en la carpeta del proyecto.

Paso 6. Escribe `node actualizar.js` y pulsa Intro.

ACTUALIZAR.JS

```
const { MongoClient } = require("mongodb");
const cliente = new MongoClient("mongodb://127.0.0.1:27017");

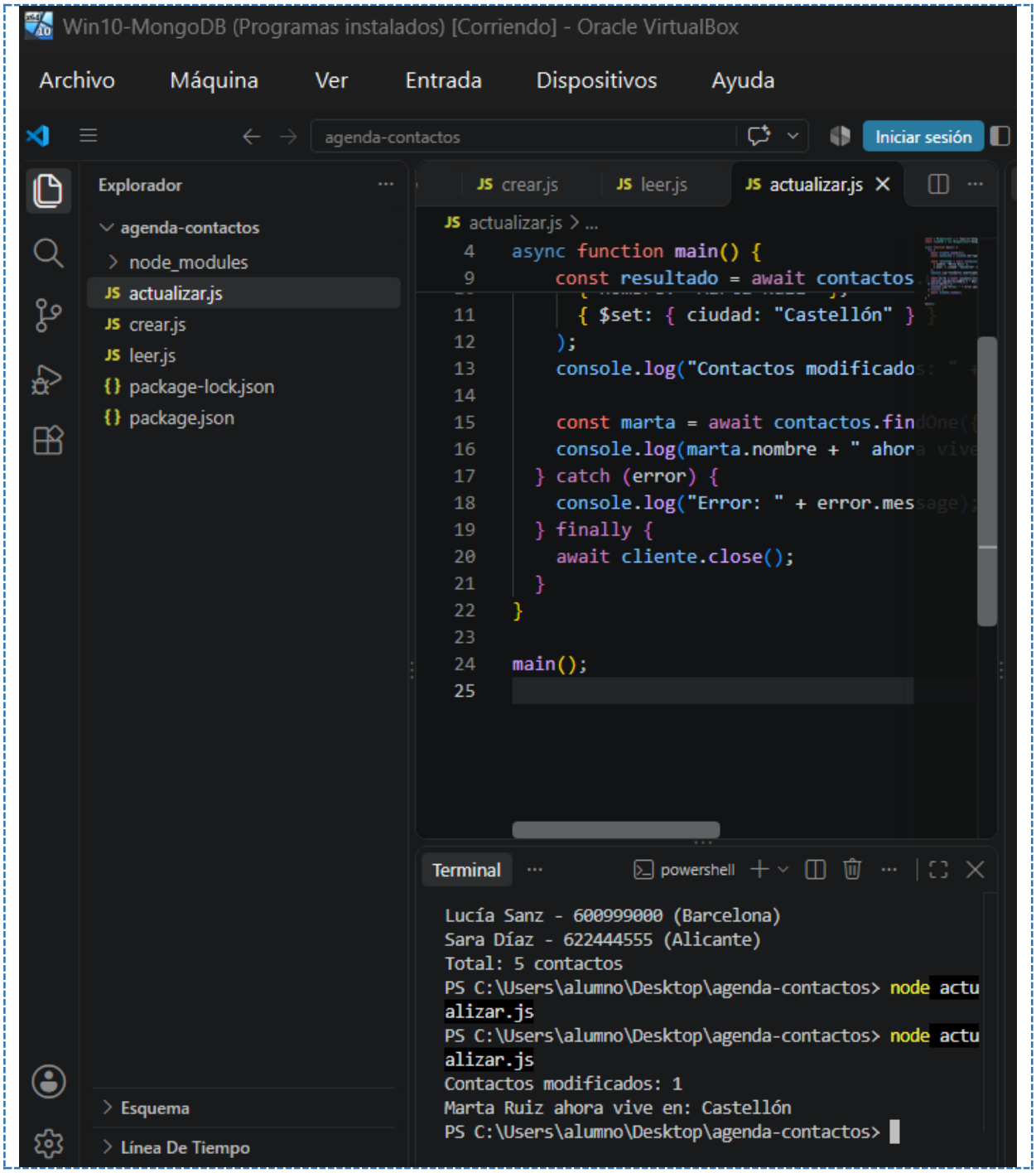
async function main() {
  try {
    await cliente.connect();
    const contactos = cliente.db("agenda").collection("contactos");

    const resultado = await contactos.updateOne(
      { nombre: "Marta Ruiz" },
      { $set: { ciudad: "Castellón" } }
    );
    console.log("Contactos modificados: " + resultado.modifiedCount);

    const marta = await contactos.findOne({ nombre: "Marta Ruiz" });
    console.log(marta.nombre + " ahora vive en: " + marta.ciudad);
  } catch (error) {
    console.log("Error: " + error.message);
  } finally {
    await cliente.close();
  }
}

main();
```

Línea	Qué hace
`contactos.updateOne( filtro, cambio )`	Igual que en mongosh: primero **a quién** cambiar y después **qué** cambiar con `$set`.
`resultado.modifiedCount`	Número de contactos modificados. Debe ser `1`.
`contactos.findOne(...)`	Busca un solo contacto y lo guarda en la variable `marta`.
`marta.ciudad`	Muestra la ciudad ya cambiada.



B.6 Borrar (D): borrar.js

Ejercicio 12 borrar.js

Crea el programa `borrar.js` que borre a **Sara Díaz** de la agenda y muestre cuántos contactos se han borrado y cuántos quedan (deben quedar **4**).

Después ejecuta otra vez `node leer.js` para ver la agenda final. Haz una captura donde se vean las dos ejecuciones.

Paso 1. En VS Code, en el panel izquierdo (Explorador), pasa el ratón por encima del nombre de la carpeta **AGENDA-CONTACTOS** y pulsa el botón **Nuevo archivo** (o usa el menú **Archivo > Nuevo archivo de texto**).

Paso 2. Escribe el nombre `borrar.js` y pulsa Intro. El archivo se abre vacío en el centro de la pantalla.

Paso 3. Escribe (o copia y pega) el código completo que aparece debajo.

Paso 4. Guarda el archivo con **Ctrl + S**. Si en la pestaña del archivo aparece un punto blanco, es que aún no está guardado.

Paso 5. Abre la terminal de VS Code con **Ctrl + ñ** (o menú **Terminal > Nuevo terminal**). Ya está situada en la carpeta del proyecto.

Paso 6. Escribe `node borrar.js` y pulsa Intro.

Paso 7. Escribe `node leer.js` y pulsa **Intro** para comprobar que Sara Díaz ya no está y que Marta Ruiz aparece en Castellón.

BORRAR.JS

```
const { MongoClient } = require("mongodb");
const cliente = new MongoClient("mongodb://127.0.0.1:27017");

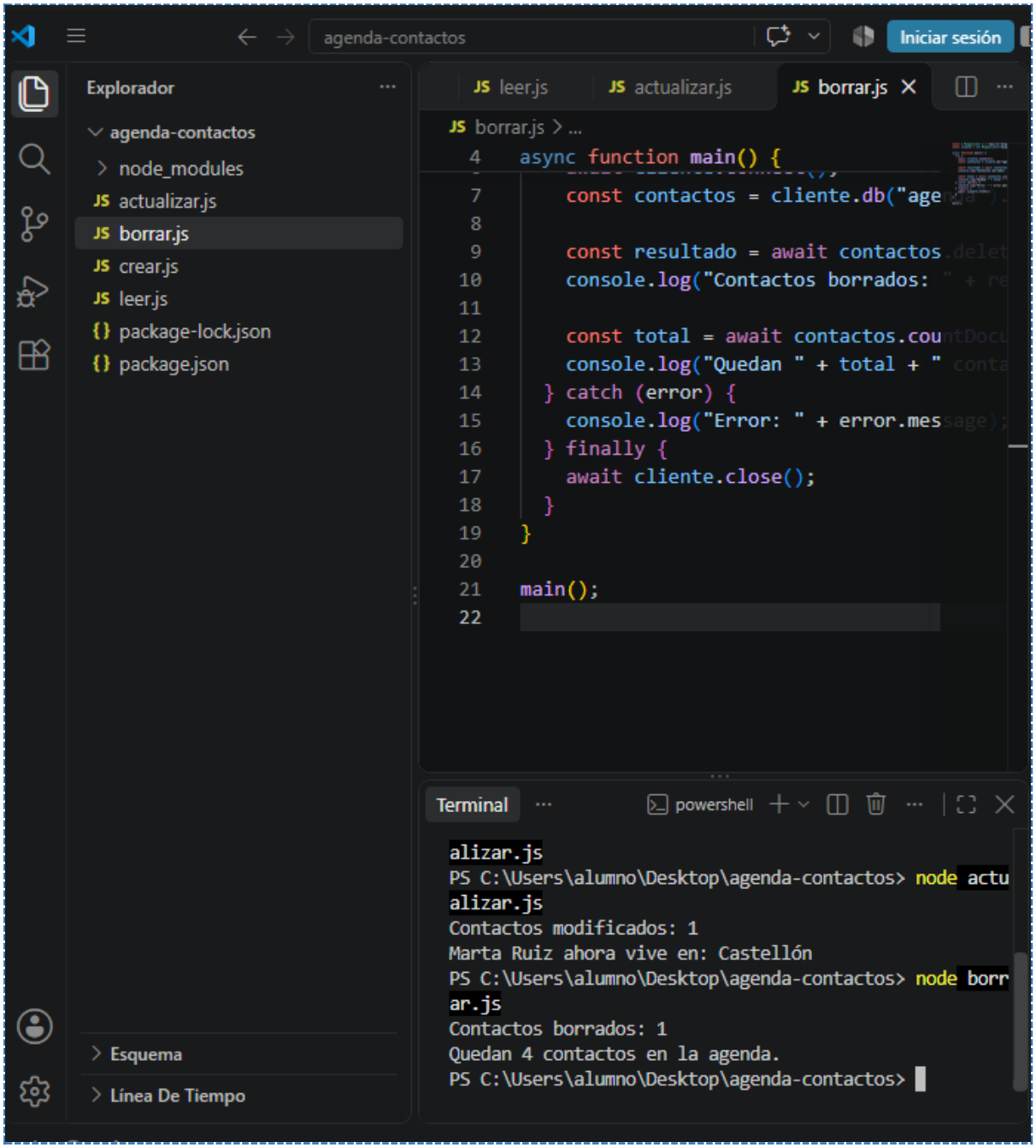
async function main() {
  try {
    await cliente.connect();
    const contactos = cliente.db("agenda").collection("contactos");

    const resultado = await contactos.deleteOne({ nombre: "Sara Díaz" });
    console.log("Contactos borrados: " + resultado.deletedCount);

    const total = await contactos.countDocuments();
    console.log("Quedan " + total + " contactos en la agenda.");
  } catch (error) {
    console.log("Error: " + error.message);
  } finally {
    await cliente.close();
  }
}

main();
```

Línea	Qué hace
`contactos.deleteOne({ nombre: "Sara Díaz" })`	Borra el primer contacto que se llame Sara Díaz.
`resultado.deletedCount`	Número de contactos borrados. Debe ser `1`.
`contactos.countDocuments()`	Cuenta los contactos que quedan. Debe ser `4`.



B.7 Sacar el proyecto de la máquina virtual

Para entregar el proyecto hay que copiarlo a la carpeta compartida **sin** `node_modules`, que ocupa mucho y se puede volver a generar con `npm install`.

Paso 1. Abre un **Símbolo del sistema** nuevo dentro de la máquina virtual.

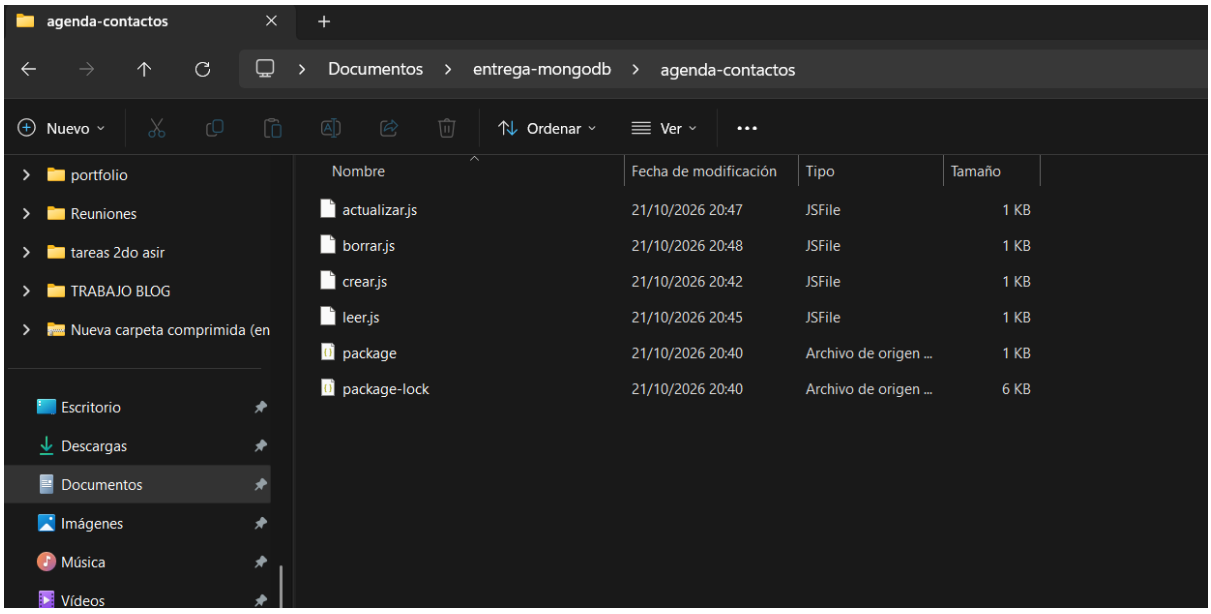
Paso 2. Escribe la orden de debajo en una sola línea y pulsa **Intro**.

Paso 3. En tu ordenador real, dentro de la carpeta `entrega-mongodb`, aparecerá la carpeta `agenda-contactos` con los archivos `crear.js`, `leer.js`, `actualizar.js`, `borrar.js`, `package.json` y `package-lock.json`.

> **ÓRDENES**
>
> ```
> robocopy %USERPROFILE%\Desktop\agenda-contactos Z:\agenda-contactos /E /XD
> node_modules
> ```

- `/E` copia también las subcarpetas y `/XD node_modules` deja fuera esa carpeta.
- Si la carpeta compartida tiene otra letra en **Este equipo**, cambia `Z:` por esa letra.

EN EL ANFITRION ES ESTO:



5. Lista de comprobación final

Antes de entregar, revisa que tienes todo:

Ejercicio	Qué debe verse en la captura	Hecho
1	Versiones de Node.js, npm y MongoDB	
2	Base de datos `agenda` y el contacto de Ana López	
3	Los cuatro contactos insertados con `insertMany`	
4	Los cinco contactos y el número 5	
5	Los dos contactos de Valencia	
6	Luis Martín con el teléfono 611222333	
7	Pedro Gómez borrado y 4 contactos en total	
8	Proyecto con `package.json` y `node_modules`	
9	`crear.js`: Sara Díaz creada y 5 contactos	
10	`leer.js`: lista de los cinco contactos	
11	`actualizar.js`: Marta Ruiz en Castellón	
12	`borrar.js`: Sara Díaz borrada y 4 contactos	
Entrega	Proyecto copiado a la carpeta compartida sin `node_modules`	

Para terminar, apaga la máquina virtual desde **Inicio > Inicio/Apagado > Apagar**.